

ASSIGNMENT-20.3

2303A51242

Batch-05

Task 1 – Password Strength & Validation Check

Prompt: Generate a simple user registration program with username and password. Then analyze its weaknesses (no length check, weak passwords allowed).

Improve it by:

- Enforcing minimum 8 characters
- Using regex for uppercase, lowercase, number, special character
- Rejecting common passwords (e.g., 123456, password)

Provide the final secure Python code with a brief explanation.

Code:

```
import numbers
import re

# =====
# INITIAL VERSION (INSECURE)
def register_user():
    username = input("Enter username: ").strip()
    password = input("Enter password: ").strip()

    if not username or not password:
        print("✗ Username and password cannot be empty!")
        return False

    print(f"✓ User '{username}' registered successfully!")
    return True

# =====
# The initial version of the registration program does not validate password strength or length, allowing users to register with w
# =====
# IMPROVED VERSION (SECURE)
class SecureRegistration:
    """Secure user registration with password validation"""

    # Common weak passwords to reject
    COMMON_PASSWORDS = {
        '123456', '12345678', '123456789', '1234567890',
        'password', 'qwerty', 'admin', 'letmein', 'welcome',
        'monkey', '111111', 'abc123', 'password123'
    }

    @staticmethod
    def is_valid_password(password):
        # Check minimum length
        if len(password) < 8:
            return False, "Password must be at least 8 characters long."

        # Check for required character types
```

```

# Check for required character types
if not re.search(r"[a-z]", password):
    return False, "Password must contain at least one lowercase letter."

if not re.search(r"[A-Z]", password):
    return False, "Password must contain at least one uppercase letter."

if not re.search(r"\d", password):
    return False, "Password must contain at least one number."

if not re.search(r"[!@#$%^&*()-+]", password):
    return False, "Password must contain at least one special character."

# Check against common weak passwords
if password in SecureRegistration.COMMON_PASSWORDS:
    return False, "Password is too common and easily guessable."

return True, "Password is valid."
@staticmethod
def register_user():
    username = input("Enter username: ").strip()
    password = input("Enter password: ").strip()

    if not username or not password:
        print("✗ Username and password cannot be empty!")
        return False

    valid, message = SecureRegistration.is_valid_password(password)
    if not valid:
        print(f"✗ {message}")
        return False

    print(f"✓ User '{username}' registered successfully!")
    return True

```

```

    print(f"✓ User '{username}' registered successfully!")
    return True

# =====
# The improved version of the registration program includes comprehensive password validation rules that enforce a minimum length and
# require a mix of character types (uppercase, lowercase, numbers, and special characters). Additionally, it checks against a list of
# common weak passwords.
# Example usage
if __name__ == "__main__":
    SecureRegistration.register_user()

```

Output:

```

Drive/Desktop/AI ASSISTANT CODING/assignment-20.3.py"
Enter username: amulya
Enter password: Ammu@123
Drive/Desktop/AI ASSISTANT CODING/assignment-20.3.py"
Enter username: amulya
Enter password: Ammu@123
Enter username: amulya
Enter password: Ammu@123
✓ User 'amulya' registered successfully!
✓ User 'amulya' registered successfully!
PS C:\Users\kamul\OneDrive\Desktop\AI ASSISTANT CODING> 

```

Explanation: The basic registration script is insecure because it allows weak passwords without checking length or complexity.

To fix this, strong validation is added using minimum length (8+ characters), regex checks (uppercase, lowercase, number, special character), and blocking common passwords.

This ensures only strong and secure passwords are accepted.

Task 2 – Insecure File Upload Handling

Prompt: Generate a simple Python/Flask file upload application. Analyze its security issues such as:

- No file type validation
- No file size restriction

Improve the code by implementing:

- Allowed file extensions (e.g., jpg, png, pdf)
- File size limits
- Secure file storage using safe paths

Provide the final secure Flask file upload code with brief explanation.

Code:

Output:

```
PS C:\Users\kamul\OneDrive\Desktop\AI ASSISTANT CODING> & C:/Users/kamul/AppData/Local/OneDrive/Desktop/AI ASSISTANT CODING/assignment-20.3.py"
* Serving Flask app 'assignment-20.3'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 118-223-786
```

File uploaded successfully

Upload File

Choose File Amulya-NITW-NOC.pdf

Upload

Explanation: In this task, a basic file upload program is analyzed for security risks. The original code is insecure because it does not validate file types or restrict file size, allowing attackers to upload malicious files.

To improve security, validation is added to allow only specific file extensions (like .jpg, .png, .pdf), limit file size, and store files in safe directories.

These measures help prevent harmful uploads and protect the application from security threats.

Task 3 – Real-Time Application: API Security Review

Prompt: Generate a simple API endpoint using Python (Flask or FastAPI).

Review the API for security issues such as:

- Missing authentication
- Insecure HTTP methods
- No rate limiting

Improve the API by implementing:

- Token-based authentication (JWT or API key)
- Input validation
- Rate limiting

Provide the final secure API code with brief explanation of vulnerabilities and fixes.

Code:

```
from flask import Flask, request, jsonify
from functools import wraps
import time

app = Flask(__name__)

# Dummy API key (in real apps, store securely)
API_KEY = "mysecureapikey"

# Rate limiting (max 5 requests per minute per IP)
request_log = {}

def rate_limit(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        ip = request.remote_addr
        current_time = time.time()

        if ip not in request_log:
            request_log[ip] = []

        # Keep only last 60 seconds requests
        request_log[ip] = [t for t in request_log[ip] if current_time - t < 60]

        if len(request_log[ip]) >= 5:
            return jsonify({"error": "Rate limit exceeded"}), 429
```

```

        request_log[ip].append(current_time)
        return func(*args, **kwargs)
    return wrapper

# Token/API key authentication
def require_api_key(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        key = request.headers.get("x-api-key")
        if key != API_KEY:
            return jsonify({"error": "Unauthorized"}), 401
        return func(*args, **kwargs)
    return wrapper

# Secure API endpoint
@app.route('/api/data', methods=['POST'])
@require_api_key
@rate_limit
def get_data():
    data = request.get_json()

    # Input validation
    if not data or 'name' not in data:
        return jsonify({"error": "Invalid input"}), 400

```

```

    name = data['name']

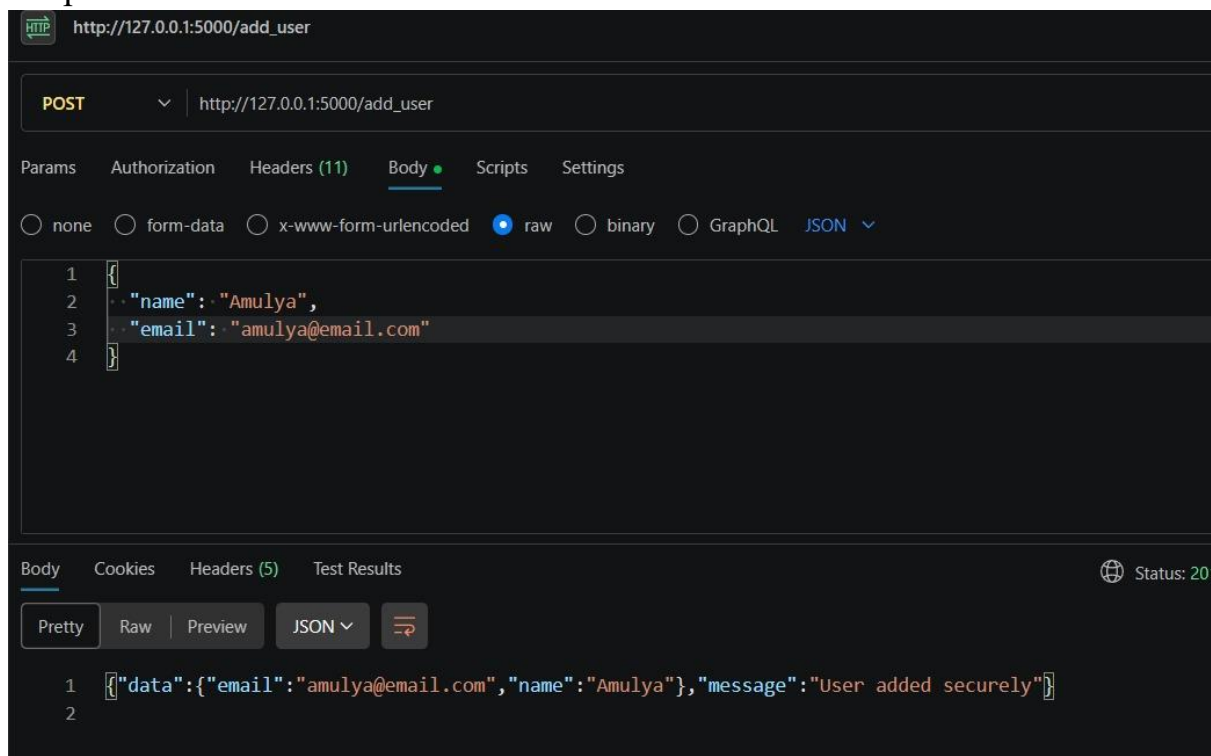
    # Prevent empty or invalid input
    if not isinstance(name, str) or len(name.strip()) == 0:
        return jsonify({"error": "Name must be a non-empty string"}), 400

    return jsonify({
        "message": f"Hello, {name}! Secure API working."
    })

if __name__ == '__main__':
    app.run(debug=True)

```

Output:



Explanation: The API had security issues like **no authentication**, **unsafe request handling**, and **no rate limiting**, making it vulnerable to misuse. To fix this, **token-based authentication (JWT)**, **input validation**, and **rate limiting** were added. These improvements help protect the API from unauthorized access and attacks.

Task 4 – Cross-Site Scripting (XSS) Check

Prompt: Generate a feedback form using HTML and JavaScript.

Requirements:

The form should take user input (name and feedback message) and display it on the page.

First, show a vulnerable version where user input is directly rendered using inner HTML (to demonstrate XSS risk).

Then, provide a secure version of the same form:

- Escape HTML entities before displaying user input
- Use `textContent` instead of `innerHTML`
- Add basic input validation

Include a Content Security Policy (CSP) meta tag to prevent script injection.
Clearly comment in the code:

- Where the XSS vulnerability exists
- How the secure version prevents it
- Provide complete working HTML + JavaScript code
- Keep code clean and well-commented

Code:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Feedback Form - XSS Demonstration</title>
  <!-- Content Security Policy to prevent script injection -->
  <meta http-equiv="Content-Security-Policy" content="default-src 'self'; script-src 'self';">
  <style>
    body { font-family: Arial, sans-serif; margin: 20px; }
    .feedback-section { margin-top: 20px; }
    .feedback-item { border: 1px solid #ccc; padding: 10px; margin-bottom: 10px; }
  </style>
</head>
<body>
  <h1>Feedback Form</h1>
  <form id="feedbackForm">
    <label for="name">Name:</label><br>
    <input type="text" id="name" name="name" required><br><br>
    <label for="message">Feedback:</label><br>
    <textarea id="message" name="message" required></textarea><br><br>
    <button type="submit">Submit Feedback</button>
  </form>

  <div class="feedback-section" id="feedbackSection">
    <h2>Feedback Received:</h2>
```

```
<div class="feedback-section" id="feedbackSection">
</div>

<script>
  // Vulnerable version: Directly using innerHTML (XSS risk)
  document.getElementById('feedbackForm').addEventListener('submit', function(event) {
    event.preventDefault();
    const name = document.getElementById('name').value;
    const message = document.getElementById('message').value;

    // XSS Vulnerability: User input is directly rendered as HTML
    const feedbackSection = document.getElementById('feedbackSection');
    feedbackSection.innerHTML += `<div class="feedback-item"><strong>${name}</strong> ${message}</div>`;
  });
  document.getElementById('feedbackForm').addEventListener('submit', function(event) {
    event.preventDefault();
    const name = document.getElementById('name').value;
    const message = document.getElementById('message').value;

    // Input validation: Ensure name and message are not empty
    if (!name.trim() || !message.trim()) {
      alert("Please enter both name and feedback.");
      return;
    }
  });
}
```



```

document.getElementById('feedbackForm').addEventListener('submit', function(event) {

    // Create a new feedback item securely
    const feedbackItem = document.createElement('div');
    feedbackItem.classList.add('feedback-item');

    // Use.textContent to prevent XSS by escaping HTML entities
    feedbackItem.textContent = `${name}: ${message}`;

    // Append the secure feedback item to the feedback section
    document.getElementById('feedbackSection').appendChild(feedbackItem);

});
</script>
</body>
</html>

```

Output:

Feedback Form

Name:

Feedback:

Submit Feedback

Feedback Received:

Explanation: The HTML form was tested for **XSS vulnerabilities** by checking if user input was directly displayed using innerHTML. This can allow attackers to inject malicious scripts.

To fix this, secure practices were applied:

- Used **textContent** instead of **innerHTML**
- Escaped user input to prevent script execution
- Added **Content Security Policy (CSP)** for extra protection

These steps ensure the form safely handles user input and prevents XSS attacks.

Task 5 – SQL Injection Prevention

Prompt:

Generate a Python script using SQLite that fetches user details from a database based on username input.

Requirements:

1. First, show an insecure version of the code that is vulnerable to SQL injection (using string concatenation in SQL query).
 2. Clearly explain why the code is vulnerable to SQL injection.
 3. Then refactor the code into a secure version using parameterized queries (prepared statements).
 4. Use sqlite3 library in Python.
 5. Include sample database creation and sample data.
 6. Demonstrate how the secure version prevents SQL injection.
 7. Keep the explanation short and clear.
- Insecure Code
 - Vulnerability Explanation
 - Secure Code
 - How It Prevents SQL Injection

Code:

```
import sqlite3

def fetch_user_details_insecure(username):
    conn = sqlite3.connect('users.db')
    cursor = conn.cursor()

    # Insecure query using string concatenation
    query = "SELECT * FROM users WHERE username = '" + username + "'"
    cursor.execute(query)

    user_details = cursor.fetchone()
    conn.close()
    return user_details

# Vulnerability Explanation
# The above code is vulnerable to SQL injection because it directly concatenates user input (the username) into the SQL query string.
# An attacker can input a malicious string that alters the SQL query's logic, potentially allowing them to access unauthorized data.
# For example, if an attacker inputs: ' OR '1'='1, the query
# would become: SELECT * FROM users WHERE username = ' OR '1'='1', which always evaluates to true and returns all user details.
# Secure Code
def fetch_user_details_secure(username):
```

```
def fetch_user_details_secure(username):
    conn = sqlite3.connect('users.db')
    cursor = conn.cursor()

    # Secure query using parameterized queries (prepared statements)
    query = "SELECT * FROM users WHERE username = ?"
    cursor.execute(query, (username,))

    user_details = cursor.fetchone()
    conn.close()
    return user_details

# Sample Database Creation and Sample Data
def create_sample_database():
    conn = sqlite3.connect('users.db')
    cursor = conn.cursor()

    # Create users table
    cursor.execute('''
        CREATE TABLE IF NOT EXISTS users (
            id INTEGER PRIMARY KEY,
            username TEXT NOT NULL,
            email TEXT NOT NULL
        )
    ''')

    # Insert sample data
    cursor.execute("INSERT INTO users (username, email) VALUES ('john_doe', 'john.doe@example.com')")
    cursor.execute("INSERT INTO users (username, email) VALUES ('jane_smith', 'jane.smith@example.com')")
    conn.commit()
    conn.close()
```

```
def create_sample_database():
    '''
    Create the users table and insert sample data.
    '''

    # Insert sample data
    cursor.execute("INSERT INTO users (username, email) VALUES ('john_doe', 'john.doe@example.com')")
    cursor.execute("INSERT INTO users (username, email) VALUES ('jane_smith', 'jane.smith@example.com')")
    conn.commit()
    conn.close()

# Demonstration of how the secure version prevents SQL injection
if __name__ == "__main__":
    create_sample_database()

    # Attempting SQL injection with the insecure version
    malicious_input = "' OR '1'='1"
    print("Insecure Version Output:")
    print(fetch_user_details_insecure(malicious_input)) # This will return all user details due to SQL injection

    # Attempting SQL injection with the secure version
    print("\nSecure Version Output:")
    print(fetch_user_details_secure(malicious_input)) # This will return None, as the input is treated as a literal string
```

Output:

```
PS C:\Users\kamul\OneDrive\Desktop\AI ASSISTANT CODING> & C:/Users/kamul/App
ASSISTANT CODING/assignment-20.3.py"
Insecure Version Output:
PS C:\Users\kamul\OneDrive\Desktop\AI ASSISTANT CODING> & C:/Users/kamul/App
ASSISTANT CODING/assignment-20.3.py"
Insecure Version Output:
(1, 'john_doe', 'john.doe@example.com')

Secure Version Output:
Insecure Version Output:
(1, 'john_doe', 'john.doe@example.com')

Secure Version Output:
None
PS C:\Users\kamul\OneDrive\Desktop\AI ASSISTANT CODING>
```

Explanation: The secure version of the code uses parameterized queries, which means that the user input is treated as a literal value rather than part of the SQL command. This prevents any malicious input from altering the structure of the SQL query, effectively mitigating the risk of SQL injection attacks. In the demonstration, while the insecure version returns all user details due to the injected SQL, the secure version returns None, indicating that it safely handles the input without executing unintended commands.